



Adji-SP

Building a CSV Profiling and ESP32 OTA Dashboard

A friendly, hands-on guide from first click to first update

FIRST EDITION

CSV_PROFILING



Building a CSV Profiling and ESP32 OTA Dashboard

First edition, September 2026.

This handbook documents the local project as implemented in this repository. The supplied *beautybook* PDF inspired its visual system. Its text and examples were written specifically for CSV_PROFILING.

Commands that affect hardware are marked clearly. Review the ESP32-S3 flash size, serial port, partition table, and local network settings before using them.



PREFACE

Open the project folder and it can feel as though three different subjects have landed on the same desk: data analysis, web development, and embedded Rust. That feeling is normal. We will pick up one piece at a time.

The dashboard has two working rooms behind it. Flask and Pandas look after CSV files. Axum looks after firmware builds and devices. Farther down the line, an ESP32-S3 calls home over Wi-Fi and asks whether a new application is waiting. Once you can picture those rooms, the source tree stops looking like a wall of unfamiliar names.

This book is meant to be used with the project open beside it. Read a few pages, touch the file being discussed, and try the small checkpoint at the end of the chapter. Keep the terminals where you can see them. A failed command often tells you exactly which door is still locked.

The CSV_PROFILING project

2026



HOW TO USE THIS BOOK

Treat these pages like a marked-up workshop notebook. You do not need to memorize the code. Keep the repository open, follow the paths printed in monospace, and compare the short examples with the real files. Windows commands use PowerShell. Linux and macOS commands use a POSIX shell.

You will meet three kinds of notes in the margins of the lesson:

From the tutor's notebook

A tutor's note slows down for the question someone would usually ask across the table.

Pause before you continue

A pause note appears before a command can touch hardware, delete data, or loosen a safety boundary.

Try it before moving on

A small exercise lets your own screen confirm what the page just explained.

You can begin without an ESP32 on the desk. The dashboard, APIs, database, ZIP checks, and error messages all run on the computer. When the lesson reaches a physical flash, the page will say so plainly.

CONTENTS

I	Foundations	1
1	Meet the Project Before We Touch It	3
1.1	One dashboard, two helpers	3
1.2	Follow a CSV file with your finger	3
1.3	Now follow a firmware ZIP	4
1.4	Use the folders as signposts	4
1.5	Which main program is the real main program?	5
1.6	Why the first flash uses USB	5
1.7	Who gets the Wi-Fi modem?	5
2	Let's Get It Running	6
2.1	Begin with the computer	6
2.2	Prepare Python gently	6
2.3	Check the Rust tools you already have	6
2.4	Make your private .env file	7
2.5	Press the project's start button	7
2.6	Ask each service a tiny question	8
2.7	If the page refuses to open	8
II	The Dashboard and CSV Profiler	9
3	Walk Through the Dashboard	11
3.1	Start with what your eyes can see	11
3.2	Meet the frontend files one by one	11
3.3	A button is a short chain of events	11
3.4	Why there are two API addresses	12
3.5	Watch the OTA build page change mood	12
3.6	Understand the device cards	13

3.7	Make one harmless visual change	13
4	Follow One CSV File	14
4.1	Make a tiny file we can understand by sight	14
4.2	The upload route receives the envelope	14
4.3	Progress arrives like notes under a door	15
4.4	Where the report sleeps	15
4.5	Clean a copy, then look again	15
4.6	Use the playground as a scratch pad	16
4.7	How to add a CSV feature without hurting OTA	16
III	The Rust OTA Service	17
5	Meet the Rust OTA Server	19
5.1	Read main.rs as a morning routine	19
5.2	Configuration is the page of house rules	19
5.3	Routes are the labels on the front desk	20
5.4	Why the browser is allowed through	20
5.5	Errors should speak in complete sentences	21
5.6	SQLite is the server's notebook	21
6	What Really Happens to the ZIP	22
6.1	Begin at the upload form	22
6.2	The server checks the suitcase before unpacking it	22
6.3	The project chooses one of two paths	22
6.4	How two files named main.rs live together	23
6.5	The builder checks its tools	23
6.6	Cargo turns source into an ELF	23
6.7	esptool prepares the OTA image	24
6.8	The fingerprint closes the package	24
6.9	Read the build states as a little story	24
IV	Firmware on the ESP32-S3	26
7	The Program That Lives on the ESP32	28

7.1	Give the board a dependable base	28
7.2	The first journey goes through USB	28
7.3	Read the true fn main like a story	29
7.4	Open the smaller crates	29
7.5	AppContext is the handover bag	30
7.6	A new flash does not stack old code	30
8	Write Your First Uploaded Firmware	31
8.1	Copy the small example before starting from nothing	31
8.2	Your program still feels like main.rs	31
8.3	The value between setup and main is your toolbox	32
8.4	Take a peripheral during setup	32
8.5	Add ordinary Cargo libraries	33
8.6	Give the release a useful version	33
8.7	Zip the contents, not the outer shelf	33
9	Watch an OTA Update Travel	35
9.1	The dashboard leaves a note for the device	35
9.2	The device introduces itself	35
9.3	The update check returns a small manifest	36
9.4	The binary moves one cup at a time	36
9.5	The device narrates the update	36
9.6	Look inside the two-drawer partition table	37
9.7	Rollback gives a new image a trial period	37
V	Practice and Extension	38
10	Put Everything Together	40
10.1	Clear a little space on the desk	40
10.2	Stage one: wake both PC services	40
10.3	Stage two: give the old CSV path a quick health check	40
10.4	Stage three: pack the basic application	41
10.5	Stage four: inspect what a successful build made	41
10.6	Stage five: give the board its first base program	41
10.7	Stage six: leave the update note	42

10.8 Stage seven: change something you can recognize	42
10.9 Write down the evidence while it is fresh	42
11 When Something Goes Wrong	43
11.1 Sit with the last thing that worked	43
11.2 The dashboard page will not open	43
11.3 CSV works and OTA says offline	43
11.4 The ZIP is turned away	44
11.5 The build stops before your code appears	44
11.6 Rust gives you a page of compiler errors	44
11.7 Cargo succeeds and image creation fails	44
11.8 The PC sees firmware and the board does not	45
11.9 The device card never appears	45
11.10The card stays on Pending	45
11.11Size or SHA-256 does not match	45
11.12The binary has outgrown its drawer	45
11.13Delete answers with 409 Conflict	45
11.14Where are the ESP32 terminal messages?	46
12 Where You Can Take It Next	47
12.1 Choose one seam and keep the rest quiet	47
12.2 Welcome another ESP32 chip	47
12.3 Turn Wi-Fi setup into a friendly first-run flow	47
12.4 Bring the ESP32 terminal into the dashboard	48
12.4.1 Road one: a serial bridge on the PC	48
12.4.2 Road two: logs sent over Wi-Fi	48
12.5 Move untrusted builds into a safer room	48
12.6 Give firmware a real signature	48
12.7 Let the API grow without surprising old projects	49
12.8 Test from the cheap layer toward the board	49
A HTTP API Reference	50
A.1 Conventions	50
A.2 OTA endpoint index	51
A.3 Status	51
A.4 Projects and builds	51

A.5	Firmware	52
A.6	Devices and deployments	53
A.7	CSV endpoint index	54
B	Configuration, Commands, and Glossary	55
B.1	Environment variables	55
B.2	Command sheet	56
B.2.1	Start services	56
B.2.2	Run OTA server checks	56
B.2.3	Check embedded tools	56
B.2.4	Build the stable runtime without flashing	56
B.2.5	Build and flash the stable runtime	56
B.3	Glossary	57
B.4	Current implementation limits	57
	Closing Note	58

PART I

FOUNDATIONS



We begin with the view from the doorway. You will see the whole project in plain language, then start it without rushing past the small signs that tell us it is healthy.

1

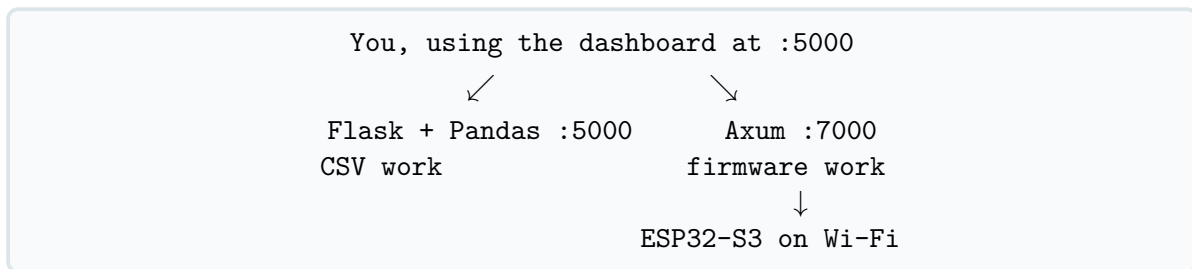
MEET THE PROJECT BEFORE WE TOUCH IT

Sec 1.1 One dashboard, two helpers

Open the dashboard in your browser. It looks like one application because every page shares the same sidebar, colors, and buttons. Behind that screen sit two separate helpers.

The Python helper listens on port 5000. Give it a CSV file and it will inspect the rows, calculate summaries, build a report, and apply cleaning choices. The Rust helper listens on port 7000. Give it a firmware project and it will guard the upload, run the build tools, save the finished binary, and remember which device should receive it.

Picture a small repair shop with one front desk and two workbenches. The front desk is the browser. One bench handles data. The other handles firmware.



Keeping the benches separate protects the CSV profiler. Adding a firmware feature does not require us to rebuild the data service in Rust. The browser only needs to know which address to call.

A name you will see often

Developers call the handoff between two programs a *service boundary*. Here the boundary is simply an HTTP request. The phrase sounds formal; the idea is ordinary. One program asks another program to do a clearly named job.

Sec 1.2 Follow a CSV file with your finger

Suppose you choose `readings.csv` and press Run Profiling. The browser sends the file to Flask. Flask saves a working copy and starts the slower profiling work in the background. While Pandas reads the data, the browser receives progress messages. When the report is ready, Flask serves the finished HTML page.

Every later CSV action returns to the same helper. Previous Runs asks Flask for saved sessions. Cleaning asks Flask to change a working copy. The playground asks Flask to perform a small Pandas operation.

You can remember the route in one line:

`browser → Flask → Pandas → report`

Sec 1.3 Now follow a firmware ZIP

The OTA journey begins with a ZIP because a Rust program is more than one source file. It may have a Cargo manifest, extra modules, and library dependencies. The browser sends that ZIP, a board choice, and a version to Axum.

Axum checks the archive before opening it. Then it asks Cargo to compile the project for the ESP32-S3. Cargo produces an ELF executable. `espflash` turns that ELF into an ESP-IDF application image. The server calculates a SHA-256 fingerprint and saves the binary under `firmware-builds`.

Later, the device asks whether an update is waiting. If one is assigned, the ESP32 downloads the `.bin`. It never receives the ZIP or a loose `.rs` file.

`ZIP → Cargo → ELF → application .bin → ESP32`

Sec 1.4 Use the folders as signposts

Keep this table nearby during your first few sessions. You only need the first sentence for each folder.

Folder or file	What lives there
<code>web/</code>	The pages you see: HTML, CSS, and browser JavaScript.
<code>python/profiler_server.py</code>	Flask routes and all existing CSV work.
<code>ota-server/</code>	The Axum server, database code, ZIP checks, build runner, and device API.
<code>firmware-projects/</code>	Private working folders made when Axum extracts uploads.
<code>firmware-builds/</code>	Finished OTA application binaries.
<code>data/ota.db</code>	The SQLite notebook where Axum remembers projects, builds, firmware, devices, and deployments.
<code>examples/esp32-rust-ota-client/</code>	The steady base program installed on the board and reused for managed builds.
<code>examples/uploaded-firmware-basic/</code>	The smallest application ZIP you can study and upload.
<code>examples/uploaded-firmware-with-libs/</code>	A second application showing extra Cargo libraries.
<code>docs/OTA_GUIDE.md</code>	Shorter engineering notes about partitions, rollback, and networking.

The three storage folders fill themselves while the system runs. Your hand-written source belongs in `web`, `python`, `ota-server`, or an application project under `examples`.

Sec 1.5 Which main program is the real main program?

There are three programs running in three places. The browser runs JavaScript. The PC runs Flask and Axum. The ESP32 runs one compiled firmware image.

Inside that firmware image, `examples/esp32-rust-ota-client/src/main.rs` owns the true executable `fnmain()`. Your uploaded application also keeps a file named `src/main.rs`, but it exports two callable functions named `setup` and `main`. During the build, Rust links those functions into the stable runtime.

Think of the runtime as a house with its wiring already installed. Your uploaded application decides what happens in the rooms. The final build produces one complete house. No second program balances on top of it.

Sec 1.6 Why the first flash uses USB

A new ESP32 does not know your Wi-Fi name. It has no address for the OTA server and no two-slot OTA partition table. USB gives it that first foundation: bootloader, partitions, Wi-Fi-aware runtime, and a small default application.

After the first successful flash, the board can introduce itself to the server and ask for later releases over Wi-Fi.

The short answer

Yes, flash the stable runtime to the ESP32-S3 first. You normally do this once while provisioning the board. OTA can take over after the device can join the network and reach port 7000 on your PC.

Sec 1.7 Who gets the Wi-Fi modem?

The ESP32 has a single modem resource. The runtime takes it once and gives it to `wifi-manager`. That manager keeps the connection alive for heartbeats and OTA checks. Your application receives the network's current state through `AppContext`, along with the remaining pins and hardware controllers it may use.

This arrangement leaves room for a friendlier Wi-Fi setup screen later. Saved credentials, reconnect rules, or a captive portal can stay inside the manager. Sensor applications can remain focused on sensors.

Pause before you continue

Do not call `Peripherals::take()` inside an uploaded application. The runtime has already taken the global set. Reach for `context.peripherals` and take only the pin or controller your application needs.

Try it before moving on

Open the repository and find these four places: `web/index.html`, `python/profiler_server.py`, `ota-server/src/main.rs`, and `examples/esp32-rust-ota-client/src/main.rs`. Say aloud which machine runs each one. That small sentence prevents a great deal of confusion later.

LET'S GET IT RUNNING

Sec 2.1 Begin with the computer

Leave the ESP32 unplugged for the moment. The first win is smaller: open the dashboard and make both PC services answer. Once that works, every later problem has fewer hiding places.

You need Python 3.10 or newer for the CSV side and Rust for the OTA side. The embedded toolchain can wait until the dashboard is comfortable.

Sec 2.2 Prepare Python gently

A virtual environment gives this project its own box of Python packages. It keeps your other Python work out of the way.

Open PowerShell in the repository and type the first line:

```
1 py -3 -m venv python\.venv
```

When the command returns, look inside `python`. A new `.venv` folder should be there. Now install the packages named in `python/requirements.txt`:

```
1 python\.venv\Scripts\python -m pip install -r python\requirements.txt
```

On Linux or macOS, the same two steps are:

```
1 python3 -m venv python/.venv
2 python/.venv/bin/pip install -r python/requirements.txt
```

The installation may take a while because data-profiling libraries bring several helpers with them. Wait for the command prompt to return. Red text near the middle can be a warning; the final lines tell you whether installation finished or failed.

Sec 2.3 Check the Rust tools you already have

Run one command at a time:

```
1 cargo --version
2 rustc --version
```

If both print a version, the PC can begin building the Axum server. Firmware needs two more checks

later:

```
1 rustc +esp --version
2 espflash --version
```

Do not chase every missing embedded tool yet. The dashboard can still start, store projects, validate ZIP files, and show an honest failed-build message.

A Windows detail that surprises many people

The ESP Rust toolchain may be installed while Windows still lacks `link.exe`. That file comes with Visual Studio Build Tools and the Desktop development with C++ workload. If a build names `link.exe`, the Rust code is not necessarily at fault. The compiler is asking Windows for a native tool it cannot find.

Sec 2.4 Make your private .env file

Copy `.env.example` and name the copy `.env`. This is your local settings page. It may hold a Wi-Fi password, so Git should ignore it.

For the first start, pay attention to these lines:

```
OTA_BIND_ADDRESS=0.0.0.0
OTA_PORT=7000
OTA_PUBLIC_BASE_URL=http://192.168.1.5:7000

WIFI_SSID=your-wifi-name
WIFI_PASS=your-wifi-password
DEVICE_NAME=ESP32-S3 Managed Device
```

Replace `192.168.1.5` with the LAN address of your own PC. On Windows, run `ipconfig` and look under the Wi-Fi or Ethernet adapter you are actually using. The useful address often begins with `192.168` or `10`.

Why localhost will betray the device

On your PC, `localhost` means your PC. On the ESP32, the same word means the ESP32. Put the PC's LAN address in `OTA_PUBLIC_BASE_URL` and in the runtime's `OTA_SERVER_URL`.

The rest of `.env.example` already points to sensible local folders. Leave those values alone during your first run.

Sec 2.5 Press the project's start button

Windows users can run:

```
1 .\run-dev.bat
```

Two terminals should open. One says CSV Profiler and uses port 5000. The other compiles and starts the Rust OTA server on port 7000. The first Rust build may be quiet for a while because Cargo is preparing

dependencies.

Linux and macOS users can run:

```
1 chmod +x run-dev.sh
2 ./run-dev.sh
```

When both helpers are ready, open `http://localhost:5000`. You should see the sidebar and the CSV upload page.

Sec 2.6 Ask each service a tiny question

The dashboard itself proves Flask is answering. Give Axum a similarly small test by opening:

```
http://localhost:7000/api/ota/status
```

The browser should show JSON. Do not worry about every field yet. Find `"status": "online"` and check that `public_base_url` carries your LAN address.

```
{
  "service": "CSV_PROFILING OTA Server",
  "status": "online",
  "public_base_url": "http://192.168.1.5:7000",
  "supported_targets": ["esp32s3"]
}
```

Sec 2.7 If the page refuses to open

Do not restart everything at once. Look at one door at a time.

First, visit `http://127.0.0.1:5000`. If it also fails, read the Flask terminal. A missing Python package usually names itself there.

If the dashboard opens but its OTA light says offline, visit the status URL on port 7000. Read the Rust terminal if that address fails. Cargo may still be compiling, another program may own the port, or configuration may have stopped startup.

If both addresses work on the PC and the ESP32 later cannot connect, move your attention to the LAN address, firewall, and Wi-Fi network. That is a different door.

Open the dashboard through HTTP. Double-clicking `web/index.html` creates a `file:///` page with a different browser origin and can make good API code look broken.

Try it before moving on

Keep two tabs open. The first shows `http://localhost:5000`. The second shows Axum's status JSON. Point to the two matching terminal windows. You now have a clean starting line, even if no ESP32 is connected.

PART II

THE DASHBOARD AND CSV PROFILER

A single CSV file will be our guide. We will follow it from the upload button into Python, watch the report come back, and peek behind each dashboard screen as we go.

3

WALK THROUGH THE DASHBOARD

Sec 3.1 Start with what your eyes can see

Keep the dashboard open beside the project folder. The dark strip on the left is the sidebar. The thin bar across the top tells you which page is active. The large pale area is where the current task appears.

Click CSV Profiler, Previous Runs, OTA Firmware, Devices, and Build History. The browser does not load five separate HTML pages. It already has each section and shows one at a time.

Now open `web/index.html`. Search for `ota-section`. You will find the OTA Firmware screen. A little farther down are `devices-section` and `builds-section`. The HTML is the dashboard's floor plan.

A useful beginner habit

When a button behaves strangely, find its `id` in the HTML first. Then search that same ID in JavaScript. This is often faster than reading an entire script from the top.

Sec 3.2 Meet the frontend files one by one

Only four files need our attention:

web/index.html The objects on the page: links, headings, forms, tables, dialogs, and empty states.

web/style.css Their appearance: spacing, color, borders, cards, buttons, badges, progress bars, and the dark build terminal.

web/app.js The behavior of CSV pages.

web/ota.js The behavior of OTA Firmware, Devices, and Build History.

The HTML loads both scripts near its closing tag:

```
<script src="app.js"></script>
<script src="ota.js"></script>
```

That order lets the old CSV script keep its place. OTA behavior arrives beside it in a separate file.

Sec 3.3 A button is a short chain of events

Look at the Upload & Build button in `index.html`. Its ID is `ota-build-btn`. Search for that text in `ota.js`. You will find the event handler that reads the chosen file, target, and version.

The chain looks like this:

click → read form → call Axum → update the page

That same pattern appears throughout the dashboard. A navigation link changes visible sections. A refresh button calls a list endpoint and rebuilds a table. A deploy button opens a dialog, then sends the selected firmware ID.

You do not have to understand every line before making sense of the flow. Find the HTML object, find its listener, then follow the function it calls.

Sec 3.4 Why there are two API addresses

At the top of the scripts you will see:

```
1 const API_BASE = "http://localhost:5000";
2 const OTA_API_BASE = "http://localhost:7000";
```

These two lines are tiny signposts. CSV requests travel to Flask. Firmware and device requests travel to Axum. Do not merge the constants just because both services run on the same PC.

The OTA script wraps its repeated request work in `otaRequest`. In plain language, that helper says: send the request, check whether it succeeded, read a useful server error when it failed, and decode JSON when a body is present.

```
1 async function getDevices() {
2   return otaRequest("/api/ota/devices");
3 }
4
5 async function deployFirmware(deviceId, firmwareId) {
6   return otaRequest(`/api/ota/devices/${deviceId}/deploy`, {
7     method: "POST",
8     headers: { "Content-Type": "application/json" },
9     body: JSON.stringify({ firmware_id: firmwareId })
10  });
11 }
```

The first function is a question. The second is an instruction. Both travel through the same careful messenger.

Sec 3.5 Watch the OTA build page change mood

Choose a ZIP and press Upload & Build. Even before the compiler starts, the page moves through small states:

Idle Nothing has been submitted.

Uploading The browser is sending the archive.

Queued Axum has made a build record.

Building The tools on the PC are working.

Success A real binary and fingerprint were saved.

Failed Something stopped the build, and the log tells us where.

The frontend asks for the build record every few seconds. This is called polling. Imagine looking through the workshop window now and then instead of keeping someone on the phone for the whole build.

Compiler errors remain in the terminal panel. A toast may say the build failed, but the dark log area carries the useful details. Scroll upward to find the first clear error.

Sec 3.6 Understand the device cards

The Devices page is a view of what Axum remembers. It does not scan the room for Wi-Fi boards. A real ESP32 puts itself on that page by registering and sending heartbeats.

Each card shows its last reported version, desired version, network address, last-seen time, and OTA progress. If a board loses power, its old card may remain. The timestamp tells you how fresh the information is.

Sec 3.7 Make one harmless visual change

Open `web/index.html` and find the subtitle under OTA Firmware. Change a few words, save the file, and refresh the dashboard. If the old words remain, use a hard refresh so the browser stops using its cached copy.

Then undo your practice wording. You have just traced the simplest frontend loop: edit HTML, let Flask serve it, and let the browser draw it.

About the word terminal

The dark OTA terminal shows build output from the PC. It does not yet show messages printed by the ESP32. For now, use `espflashmonitor` for board output. Chapter 12 describes two honest ways to bring those logs into the dashboard later.

Try it before moving on

Pick the Devices navigation link. Find its ID in `index.html`, its click listener in JavaScript, the function that loads devices, and the Axum route it calls. Write those four names on paper as a small trail.

4

FOLLOW ONE CSV FILE

Sec 4.1 Make a tiny file we can understand by sight

Create `sensor-readings.csv` with these four rows:

```
device,temperature,room
alpha,24.7,lab
beta,,office
alpha,24.7,lab
gamma,31.2,workshop
```

We already know its secrets. One temperature is missing. The alpha row is repeated. The workshop is warmer than the other rooms. A tiny file lets us check whether the report is telling the truth.

Open CSV Profiler, choose the file, and press Run Profiling. Keep `python/profiler_server.py` open while the progress screen moves.

Sec 4.2 The upload route receives the envelope

Search the Python file for `@app.route("/upload")`. Flask calls the function below that line when the browser posts a file.

Read the first checks as ordinary questions:

```
1 if "file" not in request.files:
2     return jsonify({"error": "No file provided"}), 400
3
4 if not f.filename.lower().endswith(".csv"):
5     return jsonify({"error": "Only CSV files are supported"}), 400
```

If the envelope contains a CSV, Flask gives the job a random ID and saves a working copy. Your original filename goes into the session notes; the stored filename uses the random ID so two people can upload `data.csv` without choosing the same path.

The route then starts `run_profiling` on a background thread and answers the browser with the job ID.

Why answer before the report is finished?

A full profile can take time. If one browser request waits silently for the whole job, the page feels frozen. Returning the job ID gives the browser something to follow while Python continues its work.

Sec 4.3 Progress arrives like notes under a door

Every running job has a small Python queue. The profiler places progress messages into it. The route `/progress/<job-id>` waits for those messages and sends them to the browser as they arrive.

This one-way stream is called server-sent events, usually shortened to SSE. The name matters less than the picture: Python writes short notes; the browser reads them without asking the same question again and again.

```
1 def generate():
2     while True:
3         message = q.get()
4         if message is None:
5             break
6         yield message
```

When the queue receives `None`, the stream closes. The browser then opens the finished report.

Sec 4.4 Where the report sleeps

Flask stores the report HTML in `python/reports`. Beside it sits a JSON sidecar. Think of the sidecar as the label on a folder: original filename, job ID, creation time, report address, row count, and column count.

Previous Runs reads those labels through `GET/sessions`. A browser refresh does not erase the session because the important details live on disk, not only in JavaScript memory.

These routes form the small session shelf:

Method	Route	What you receive
GET	<code>/sessions</code>	All report sessions Flask can still find.
GET	<code>/session/<job-id></code>	The notes for one saved session.
GET	<code>/report/<job-id></code>	The generated HTML report.
DELETE	<code>/reset/<job-id></code>	Removal of one session and its working files.
DELETE	<code>/reset-all</code>	Removal of all local CSV sessions.

Sec 4.5 Clean a copy, then look again

Return to our four-row file. Ask the cleaning panel to remove duplicates and fill the missing temperature with the median. When you apply the choices, the browser posts a JSON description to `/clean/<job-id>`.

The idea resembles this:

```
{
  "drop_duplicates": true,
  "missing": {
    "temperature": "fill_median"
  }
}
```

Pandas opens the current working data, applies those choices, and writes `<job-id>_cleaned.csv`. The original upload remains available. Flask refreshes the report notes and gives the browser a download address.

Open the cleaned CSV. You should see three data rows and no empty temperature. Since 24.7 appears twice in the original values, the median fill should also be 24.7.

Sec 4.6 Use the playground as a scratch pad

The playground runs one small Pandas idea at a time. Start with Preview, Head, Tail, Describe, or Value Counts. Those choices do not change the saved data.

For example, this request asks for the ten rows with the largest temperature values:

```
{
  "operation": "sort_values",
  "column": "temperature",
  "limit": 10,
  "mutate": false
}
```

The word **mutate** means “change the working data.” The dashboard only sends **true** after you press the explicit mutation action for operations such as rename, fill, drop, type conversion, or setting a cell.

The route clamps the row limit between 1 and 500. That keeps an accidental giant preview from overwhelming the page.

Sec 4.7 How to add a CSV feature without hurting OTA

Put new data operations in the Flask route or a Python helper. Add the matching control to `index.html` and its behavior to `app.js`. Keep its requests on `API_BASE`.

Likewise, an OTA feature belongs in Axum and `ota.js`. The two workbenches may share the front desk, but neither needs to borrow the other’s tools.

Pause before you continue

Do not rename the existing CSV routes while adding firmware features. Saved frontend behavior already depends on those addresses. Small additions around the working flow are safer than rewriting it.

Try it before moving on

Profile the four-row file, remove its duplicate, fill the missing temperature, and download the result. Refresh the browser and recover the session from Previous Runs. Then explain why the report survives even though JavaScript memory was refreshed.

PART III

THE RUST OTA SERVICE

Now we cross into the Rust server. We will unpack one firmware ZIP carefully and follow it until a real application image and its build history appear.

MEET THE RUST OTA SERVER

Sec 5.1 Read main.rs as a morning routine

Open `ota-server/src/main.rs`. The file may look formal at first, yet its job is close to opening a workshop in the morning. Turn on the lights. Unlock the cabinets. Put the tools on the bench. Open the front door.

The Rust version of that routine goes in this order:

1. Start logging so the terminal can tell us what happens.
2. Read settings from `.env`.
3. Open the SQLite database.
4. Prepare the firmware builder.
5. Prepare the coordinator that watches each build.
6. Put those shared pieces into `AppState`.
7. Connect the HTTP routes.
8. Listen on the configured address and port.

Here is the middle of that routine with some details folded away:

```
1 let config = Arc::new(Config::load()?);
2 let storage = Storage::connect(&config.database_url()).await?;
3 let builder = Arc::new(LocalFirmwareBuilder::new(/* settings */));
4 let coordinator = Arc::new(BuildCoordinator::new(
5     storage.clone(),
6     builder,
7     config.project_dir.clone(),
8     config.build_dir.clone(),
9 ));
10
11 let state = AppState { config, storage, coordinator };
```

The word `Arc` means several requests can safely hold a shared reference to the same service object. You do not need to master it now. Notice the shape: configuration, storage, and building meet in one state bundle, and route handlers receive that bundle when they need it.

Sec 5.2 Configuration is the page of house rules

Move to `ota-server/src/config.rs`. This file reads each `OTA_...` setting, supplies local defaults, checks values, and resolves folder paths from the repository root.

It also creates `firmware-projects`, `firmware-builds`, and `data` when they are missing. That is why a

fresh copy of the project can start without you building those folders by hand.

If no public URL is supplied, the server tries to detect a likely LAN address. Detection is convenient on a simple home network. An explicit `OTA_PUBLIC_BASE_URL` is easier to reason about when a PC has Ethernet, Wi-Fi, virtual adapters, or a VPN.

When startup succeeds, the terminal prints two views:

```
OTA Server running
Local:
http://localhost:7000

LAN / public firmware URL:
http://192.168.1.5:7000
```

The local line is for the browser on the PC. The LAN line is the address an ESP32 can reach.

Sec 5.3 Routes are the labels on the front desk

Open `ota-server/src/routes/mod.rs`. This file is short because it mainly matches an address with the function that handles it.

The work is grouped into three nearby files:

routes/firmware.rs Receives project ZIP files and serves finished firmware.

routes/builds.rs Starts builds and returns build history or logs.

routes/devices.rs Remembers boards, heartbeats, deployments, update checks, and OTA progress.

Find this line:

```
.route("/api/ota/status", get(status))
```

Read it from left to right: when a GET request arrives at that address, call `status`. Other lines follow the same grammar.

Sec 5.4 Why the browser is allowed through

The page comes from port 5000 and calls a server on port 7000. Browsers treat those as different origins. Axum therefore needs a CORS rule that names the dashboard origins it trusts.

The default list contains:

```
http://localhost:5000
http://127.0.0.1:5000
```

If you serve the frontend from another address, add that exact origin to `OTA_ALLOWED_ORIGINS`. A CORS error in the browser can happen even while the Axum status URL opens normally in its own tab.

Pause before you continue

Keep the origin list narrow. Allowing every website to call a local firmware builder turns a browser convenience into a needless open door.

Sec 5.5 Errors should speak in complete sentences

Open `ota-server/src/errors.rs`. Every ordinary API error carries an HTTP status, a short machine-readable code, and a message for a person. It may also carry details and a hint.

```
{
  "success": false,
  "error": {
    "code": "INVALID_FIRMWARE_PROJECT",
    "message": "Managed application entrypoint was not found",
    "details": "src/main.rs"
  }
}
```

The frontend can show the message without guessing what went wrong. A developer can use the short code in tests. The status number gives HTTP clients the broad category:

- 400** The request could not be read.
- 404** The named record does not exist.
- 409** The request clashes with current state, such as a second active build.
- 413** The upload is too large.
- 422** The request was readable but failed a project rule.
- 500** Something inside the server failed unexpectedly.

Sec 5.6 SQLite is the server's notebook

The database file is `data/ota.db`. You do not need to create its tables. `storage.rs` does that during startup.

Each table answers a different question:

Table	The question it answers
projects	Which ZIP was uploaded, for which target and version?
builds	What happened when we tried to compile it?
firmware	Which successful binary can a device download?
devices	Which boards have introduced themselves, and what did they report?
deployments	Which firmware was assigned to which board, and how far did it get?

SQLite foreign keys tie those pages together. A firmware file that still belongs to a device or deployment cannot be casually deleted. The API returns a conflict instead of tearing a page out of the notebook and leaving dangling references behind.

Try it before moving on

Open `main.rs`, `config.rs`, `routes/mod.rs`, and `storage.rs` in four tabs. In each tab, find the one place that matches its chapter role: startup, settings, addresses, and database tables.

6

WHAT REALLY HAPPENS TO THE ZIP

Sec 6.1 Begin at the upload form

On the OTA Firmware page, the form asks for three things: a ZIP, a board, and a version. The browser packs them into a multipart request with these field names:

Field	Example	Why the server needs it
<code>project</code>	<code>sensor.zip</code>	The Rust source and Cargo files
<code>target</code>	<code>esp32s3</code>	The compiler and chip choice
<code>version</code>	<code>1.0.1</code>	The release name reported by the new firmware

Axum gives the upload a random project ID. That ID names its private extraction folder. A filename supplied by the browser never gets to choose where the server writes.

Sec 6.2 The server checks the suitcase before unpacking it

A ZIP is much like a suitcase with a list of intended destinations. A dishonest entry might claim that its file belongs in `../../somewhere-else`. Extracting that name without checking would let the archive escape its room. This attack is often called ZIP slip.

The extractor refuses parent paths, absolute paths, symbolic links, more than 10,000 entries, and more extracted data than the configured limit. It also insists on finding a `Cargo.toml`.

These checks may feel strict when your own archive is rejected. They protect every other folder on the PC. Fix the ZIP layout rather than relaxing the guard.

Sec 6.3 The project chooses one of two paths

After extraction, `services/project.rs` looks for `ota-app.toml`.

If the file exists, the project is a *managed application*. It supplies the product behavior while the stable runtime supplies Wi-Fi, OTA, and rollback.

```
schema_version = 1
application_api_version = 2
entrypoint = "src/main.rs"
```

If the file is absent, the project is *standalone firmware*. Axum builds the project as it arrived. That project

must carry its own OTA client if it should update itself again.

For a first application, managed mode is easier. You can write sensor behavior without copying the network and partition code into every upload.

Sec 6.4 How two files named `main.rs` live together

This point deserves a slow pass.

The stable runtime has a normal executable `fnmain()` in `examples/esp32-rust-ota-client/src/main.rs`. It owns boot, Wi-Fi, OTA, and application startup.

Your ZIP also keeps `src/main.rs`, shaped like an ordinary firmware program. It exports two free functions:

```
1 pub fn setup(context: &mut AppContext) -> anyhow::Result<State>;
2 pub fn main(context: AppContext, state: State) -> anyhow::Result<()>;
```

Axum copies the stable runtime into a private generated workspace. It places your application under `applications/device-app` and adjusts only the copied Cargo manifests. In that copy, Cargo treats your `src/main.rs` as a library source file. The runtime can then call `device_app::setup` and `device_app::main`.

Your uploaded files stay unchanged. Rust links both pieces into one complete image.

A kitchen picture

The stable runtime is a kitchen with power, water, and safety rules. Your upload is the recipe being cooked today. The finished meal includes both. Uploading a new recipe does not pile a second kitchen on top of the first one.

Sec 6.5 The builder checks its tools

The actual command runner lives in `services/builder.rs`. Before it touches the project, it asks three programs for their versions:

```
1 cargo --version
2 rustc --version
3 espflash --version
```

A missing tool becomes a failed build record with a clear message. Axum stays alive, so you can fix the environment and try another build.

The builder is hidden behind a small Rust trait named `FirmwareBuilder`. Today the implementation runs tools directly on the PC. Tomorrow another implementation could run them inside a locked-down container while the routes remain the same.

Sec 6.6 Cargo turns source into an ELF

The build command is:

```
1 cargo build --release --target xtensa-esp32s3-espidf
```

Axum does not build a shell sentence from uploaded text. It starts `cargo` and passes each argument separately. The working directory is the trusted project workspace.

While Cargo works, the server reads both output streams line by line and saves them in the build record. This is why the dashboard can show the real compiler message and why an old failed build still has useful logs after a browser refresh.

Sec 6.7 espflash prepares the OTA image

Cargo's ELF is useful to development tools, but the inactive OTA partition needs an ESP-IDF application image. The next command follows this shape:

```
1 espflash save-image --chip esp32s3 --format esp-idf \  
2 <ELF_FILE> firmware-sensor-v1.0.1-a8c4f2.bin
```

The builder deliberately asks for an application image. It does not make a merged factory image containing bootloader and partition data. Those larger pieces belong to initial USB provisioning.

Sec 6.8 The fingerprint closes the package

The server reads the finished `.bin` and calculates SHA-256. Think of the hash as a long fingerprint. The device calculates the same fingerprint while it downloads. If the two strings differ, the bytes changed or the wrong file arrived, so the device refuses to activate it.

On success, SQLite receives the filename, byte size, target, version, project ID, build ID, timestamp, and hash. The dashboard then adds the image to the firmware table.

A fingerprint is not a signature

SHA-256 catches corruption. It does not tell the device who approved the firmware. Production publisher identity needs real signing and protected keys. The project leaves that as a visible future boundary.

Sec 6.9 Read the build states as a little story

```
uploaded → queued → building → success  
                                     ↘ failed
```

Uploaded means the suitcase passed inspection. Queued means a build record exists. Building means the tools are working. Success means the binary and metadata are real. Failed means the log holds the reason the journey stopped.

Only one queued or active build is allowed for the same project. A second click receives a conflict instead of launching two compilers into the same working folders.

Your own ZIP can still run code on the PC

Cargo may execute `build.rs`, procedural macros, and dependency build scripts. Safe extraction protects paths, not the act of compilation. Use trusted projects until the builder runs inside a proper sandbox.

Try it before moving on

Open `examples/uploaded-firmware-basic`. Check that its ZIP root would contain `Cargo.toml`, `ota-app.toml`, and `src/main.rs`. Upload it as version 1.0.1 and watch the log. A successful toolchain should produce a real binary. A missing tool should produce a real failure. Both outcomes tell the truth.

PART IV

FIRMWARE ON THE ESP32-S3

The ESP32 finally joins us. First it receives a dependable foundation over USB. After that, your smaller application code can ride inside each complete OTA release.

7

THE PROGRAM THAT LIVES ON THE ESP32

Sec 7.1 Give the board a dependable base

Take the ESP32-S3 in your hand. Before OTA can help us, this board needs a program that knows how to join Wi-Fi, find the PC, and write a new image safely. That first program is the stable runtime under `examples/esp32-rust-ota-client`.

The word *stable* does not mean it can never change. It means ordinary product updates should not have to rewrite its plumbing. It looks after:

- ESP-IDF startup and logs;
- the Wi-Fi connection;
- device identity and heartbeats;
- update checks and downloads;
- the two OTA application slots;
- hash verification, reboot, and rollback;
- the moment your uploaded application begins running.

Your application can then spend its attention on a sensor, motor, display, or other product job.

Sec 7.2 The first journey goes through USB

Open PowerShell in the runtime directory. Fill in values for the network and the reachable server address:

```
1 cd examples\esp32-rust-ota-client
2 $env:WIFI_SSID="your-ssid"
3 $env:WIFI_PASS="your-password"
4 $env:OTA_SERVER_URL="http://192.168.1.5:7000"
5 $env:DEVICE_NAME="Sensor node"
```

Check the IP carefully. It belongs to the PC running Axum.

When the board, cable, and partition table are ready, this command builds, flashes, and opens the monitor:

```
1 cargo +esp run --release --target xtensa-esp32s3-espidf
```

The configured Cargo runner calls `espflash flash -monitor`. The first flash can install the bootloader and partition layout as well as the application.

Stop and identify the board

Flashing writes to physical hardware. Confirm that the connected board is an ESP32-S3, the USB cable carries data, the flash size matches the partition plan, and the selected serial port belongs to this board.

When the runtime boots, the serial terminal should show Wi-Fi activity. After it receives an IP address, it registers with Axum. Refresh Devices in the dashboard and look for the name you supplied.

Sec 7.3 Read the true fn main like a story

Open `examples/esp32-rust-ota-client/src/main.rs`. The function begins here:

```
fn main() -> Result<()> {
    esp_idf_svc::sys::link_patches();
    EspLogger::initialize_default();
    // platform setup continues...
}
```

Continue down the file slowly. The runtime takes the board's peripherals, separates the modem, starts the system event loop and NVS, then constructs `WifiManager`. It derives a device ID from the station MAC unless you supplied one.

Next it builds `AppContext`. This is the bag handed to your application. Before calling your application, it asks ESP-IDF whether the current image is new and still waiting for validation.

Then comes the handoff:

```
1 let application_state = device_app::setup(&mut context)?;
2
3 if unverified_image {
4     mark_running_image_valid()?;
5 }
6
7 // Start the OTA supervisor in its own thread.
8 device_app::main(context, application_state)
```

The real source contains fuller error handling. The shape above is the part to remember: prepare, test application setup, approve the new slot, keep OTA alive, and enter the application's main loop.

Sec 7.4 Open the smaller crates

The runtime is divided so each piece can be held in your head:

crates/application-api Defines what the application is allowed to receive.

crates/wifi-manager Owns the modem and keeps trying when the network drops.

crates/ota-runtime Talks to Axum and writes an update into the inactive slot.

applications/default-app Supplies the small behavior used during the first USB flash.

You can open one crate without reading the others. This is especially useful when a Wi-Fi problem and a sensor problem happen on the same day.

Sec 7.5 AppContext is the handover bag

The application receives this broad shape:

```
1 pub struct AppContext {
2     pub device: DeviceInfo,
3     pub network: NetworkHandle,
4     pub peripherals: ApplicationPeripherals,
5 }
```

`device` tells the application its ID, friendly name, chip, and firmware version. `network` lets it ask whether Wi-Fi is connected and which IP was last reported. `peripherals` carries the available pins and controllers.

Each hardware item is wrapped in `Option`. When your setup function takes `I2C0`, that option becomes empty. The code now has a visible record that one part of the application owns that controller.

The modem is missing from the bag on purpose. `wifi-manager` keeps it so heartbeat and OTA polling can continue while your application runs.

Would a Wi-Fi manager become a problem?

It becomes helpful when ownership stays in one place. The current manager already owns the modem, event loop, NVS-backed Wi-Fi service, and reconnect loop. A future setup access point or captive portal can grow there. Uploaded applications can keep reading the same network handle.

Sec 7.6 A new flash does not stack old code

Imagine two labeled drawers, `ota_0` and `ota_1`. The board runs one drawer while the updater writes a complete image into the other. After verification and reboot, the drawers trade roles.

The newly written image contains the runtime and the chosen application together. It replaces the inactive slot from beginning to end. Old structs, functions, and libraries do not remain loaded beside the new program.

The previous active drawer stays available for rollback until the new image proves it can start. On the next release, ESP-IDF may overwrite that older drawer. The PC can keep a long firmware history in SQLite, while the board still has only its two application slots.

Try it before moving on

Before connecting hardware, find the runtime's `fn main`, the call to `device_app::setup`, the call to `device_app::main`, and the place where the modem is separated from application peripherals. Trace them in that order with your finger.

8

WRITE YOUR FIRST UPLOADED FIRMWARE

Sec 8.1 Copy the small example before starting from nothing

Open `examples/uploaded-firmware-basic`. It contains only the pieces the server expects:

```
Cargo.toml
ota-app.toml
src/
  main.rs
```

Copy this folder somewhere safe and give the Cargo package a new name. Beginning from a working example lets you change one idea at a time.

The descriptor tells Axum how to treat the project:

```
schema_version = 1
application_api_version = 2
entrypoint = "src/main.rs"
```

Leave these values as they are for your first application.

Sec 8.2 Your program still feels like `main.rs`

Open `src/main.rs`. There is no application class and no framework trait to implement. You write two public functions.

```
1 use anyhow::Result;
2 use firmware_app_api::AppContext;
3
4 pub fn setup(context: &mut AppContext) -> Result<u64> {
5     log::info!("Hello from {}", context.device.device_id);
6     Ok(0)
7 }
8
9 pub fn main(context: AppContext, mut count: u64) -> Result<()> {
10     loop {
11         count += 1;
12         log::info!(
13             "count={count}, wifi={}",
14             context.network.is_connected()
15         );
16         std::thread::sleep(std::time::Duration::from_secs(10));
17     }
18 }
```

`setup` runs once after the runtime has prepared the board. `main` is your long-running loop. The stable entrypoint calls both for you.

Change the hello message. That one line is enough for a first personalized build.

Sec 8.3 The value between `setup` and `main` is your toolbox

The example returns a number from `setup` and receives it as `count` in `main`. That number is the application state.

Your own state can be as small as `()` when nothing needs to be carried. It can also be a struct containing a sensor driver, counters, buffers, or display state.

```

1 pub struct State {
2     sample_number: u64,
3     alarm_enabled: bool,
4 }
5
6 pub fn setup(context: &mut AppContext) -> Result<State> {
7     log::info!("Preparing peripherals for {}", context.device.name);
8     Ok(State {
9         sample_number: 0,
10        alarm_enabled: true,
11    })
12 }
13
14 pub fn main(context: AppContext, mut state: State) -> Result<()> {
15     loop {
16         state.sample_number += 1;
17         // Read the sensor and do your real work here.
18     }
19 }
```

This is ordinary Rust data, not a class required by the OTA system.

Sec 8.4 Take a peripheral during setup

Suppose the application needs I2C0 and some pins. The beginning may look like this:

```

1 use anyhow::{Context, Result};
2
3 pub fn setup(context: &mut AppContext) -> Result<State> {
4     let pins = context.peripherals.pins.take()
5         .context("GPIO pins were already taken")?;
6     let i2c0 = context.peripherals.i2c0.take()
7         .context("I2C0 was already taken")?;
8
9     // Construct the ESP-IDF I2C driver and sensor here.
10    // Return the finished driver inside State.
11 }
```

The calls to `take()` move those resources into your code. If `setup` is called with a missing resource, it returns a readable error. On a brand-new OTA image, that error can trigger rollback.

Keep fallible driver construction and a short startup check in `setup`. Do not let `setup` loop forever. The

runtime needs a clear success or failure before it can approve the new slot.

Pause before you continue

Return owned state from setup. A reference borrowed from `AppContext` cannot safely outlive the mutable setup call when the context later moves into main. Real ESP-IDF drivers may need careful lifetime and type aliases; follow the driver crate's example for your exact chip and pins.

Sec 8.5 Add ordinary Cargo libraries

Yes, your uploaded application can use libraries. Open `examples/uploaded-firmware-with-libs/Cargo.toml`:

```
[dependencies]
anyhow = "1"
firmware-app-api = "0.1"
heapless = "0.8"
log = "0.4"
serde = { version = "1", features = ["derive"] }
serde_json = "1"
```

That example keeps samples in a fixed-capacity queue and turns telemetry into JSON. Sensor drivers, `embedded-hal` helpers, compact collections, and serialization crates can work the same way.

The library must support `xtensa-esp32s3-espidf`. A desktop crate that assumes a mouse, filesystem, or full operating system may compile poorly or make little sense on the board. Check the crate's examples and features.

If you use a local path dependency, include its whole folder inside the ZIP. The path must stay under the uploaded project root. Axum rejects paths that reach outside the archive.

Why firmware-app-api looks like a normal dependency

The uploaded manifest uses `firmware-app-api="0.1"` as a portable placeholder. In its private build copy, Axum points that name to the matching API crate inside the stable runtime. Your original manifest is left alone.

Sec 8.6 Give the release a useful version

The version typed into the dashboard becomes the firmware version compiled into the managed runtime and stored in SQLite. Begin with a calm sequence such as 1.0.0, 1.0.1, and 1.1.0.

Deployment selects a specific firmware ID. The current server checks whether the desired version equals the device's current version; it does not sort semantic versions and choose one by itself.

Sec 8.7 Zip the contents, not the outer shelf

On Windows PowerShell:

```
1 Compress-Archive \
```

```
2 -Path examples\uploaded-firmware-basic\* \  
3 -DestinationPath uploaded-firmware-basic.zip
```

On Linux or macOS:

```
1 cd examples/uploaded-firmware-basic  
2 zip -r ../../uploaded-firmware-basic.zip \  
3   Cargo.toml ota-app.toml src README.md
```

Open the ZIP and check its first level. You should immediately see `Cargo.toml`, `ota-app.toml`, and `src`. Upload it on OTA Firmware. The device will receive the later `.bin`, never this ZIP.

Try it before moving on

Make a copy of the basic example. Change its package name and hello message. Let setup return a small state struct of your own. Zip the contents and upload version `1.0.1`. The upload response should say `managed_application`, API version 2, and `src/main.rs`.

WATCH AN OTA UPDATE TRAVEL

Sec 9.1 The dashboard leaves a note for the device

Pressing Deploy OTA does not force a connection into the ESP32. The dashboard tells Axum which firmware the device should want. Axum records that choice as pending.

The ESP32 checks the server on its own schedule. On the next check, Axum hands back the note. This is a pull-based update: the device opens every network conversation.

```
graph TD
    A[You press Deploy OTA] --> B[Axum records the desired firmware]
    B --> C["↓ the ESP32 asks for an update"]
    C --> D["download → check → install → reboot"]
    D --> E["the new runtime reports its version"]
```

A sleeping or temporarily offline board can catch the note later. The PC does not need to know how to open a connection through the board's firewall or changing address.

Sec 9.2 The device introduces itself

Once Wi-Fi is connected, the runtime posts a registration like this:

```
{
  "device_id": "ESP32-S3-A1B2C3D4E5F6",
  "name": "Sensor node",
  "chip": "esp32s3",
  "current_version": "1.0.0",
  "ip_address": "192.168.1.24"
}
```

That request creates the card on the Devices page. The server accepts a tidy device ID made from letters, numbers, dots, dashes, or underscores.

The runtime then sends a heartbeat during each cycle. A heartbeat is a short “I am still here” message with the current version, status, and IP. Axum updates `last_seen` each time.

An old card can remain after a board loses power. Look at `last_seen` before trusting the word Online.

Sec 9.3 The update check returns a small manifest

When nothing is assigned, the device receives:

```
{"update_available": false}
```

When version 1.0.1 is waiting, it receives a description of the exact binary:

```
{
  "update_available": true,
  "firmware_id": "c53b...",
  "version": "1.0.1",
  "target": "esp32s3",
  "size": 945332,
  "sha256": "f2534c...",
  "download_url": "http://192.168.1.5:7000/api/ota/firmware/c53b.../download"
}
```

The device checks the target, size, hash shape, and URL before opening an OTA writer. The server also prevents you from assigning ESP32-S3 firmware to a differently registered chip.

Sec 9.4 The binary moves one cup at a time

A firmware image can be much larger than the spare RAM on a small device. The client therefore uses a 4 KiB buffer.

Picture moving water from a barrel with one cup. Read a cup from HTTP. Pour it into the inactive flash slot. Add those same bytes to the running SHA-256 calculation. Repeat until the response ends. The whole barrel never has to sit in memory.

As bytes arrive, the runtime reports downloading progress in roughly five-percent steps. At the end it checks the total byte count and compares the finished fingerprint with the server's value.

If size or hash differs, the writer is aborted. The current running application remains untouched.

Sec 9.5 The device narrates the update

Status callbacks let the dashboard follow along:

```
{"status": "downloading", "progress": 35}
{"status": "verifying", "progress": 100}
{"status": "installing", "progress": 100}
{"status": "rebooting", "progress": 100}
```

A failure carries the reason:

```
{
  "status": "failed",
  "error": "SHA256 mismatch"
}
```

After reboot, the new image runs setup. If that succeeds, the runtime marks the slot valid and reports:

```
{
  "status": "success",
  "progress": 100,
  "current_version": "1.0.1"
}
```

Axum clears the desired firmware. The device will not be offered the same release on every poll.

Sec 9.6 Look inside the two-drawer partition table

The supplied 4 MiB example is:

#	Name,	Type,	SubType,	Offset,	Size
nvs,	data,	nvs,	0x9000,	0x6000	
otadata,	data,	ota,	0xf000,	0x2000	
phy_init,	data,	phy,	0x11000,	0x1000	
ota_0,	app,	ota_0,	0x20000,	0x1e0000	
ota_1,	app,	ota_1,	0x200000,	0x1e0000	

otadata remembers which application slot should boot. ota_0 and ota_1 are the two drawers. ESP-IDF chooses the inactive one; application code should not assume a fixed winner.

Sec 9.7 Rollback gives a new image a trial period

The example enables CONFIG_BOOTLOADER_APP_ROLLBACK_ENABLE=y. When an OTA image boots for the first time, its slot is unverified.

The runtime calls your setup function. A successful setup marks the slot valid. A failed setup marks it invalid and asks for a reboot. The rollback-aware bootloader can then return to the previous valid slot.

This safety net depends on the partition table, bootloader configuration, and application validation call working together. Test it with a deliberately failing setup on your own hardware before relying on it.

Pause before you continue

SHA-256 and plain HTTP are suitable for corruption checks on a trusted development LAN. A production device also needs an authenticity plan such as signed images, protected keys, Secure Boot, and an intentional transport policy.

Try it before moving on

Tell the update story without endpoint names: leave a note, let the board ask, move the image in small cups, compare fingerprints, switch drawers, test startup, and report the new version. Then match each step to the code or dashboard screen.

PART V

PRACTICE AND EXTENSION



The last part is a workbench. You will run the whole path, learn how to read common failures, and sketch improvements without pulling apart the pieces that already work.

10

PUT EVERYTHING TOGETHER

Sec 10.1 Clear a little space on the desk

This is the day we run the whole path. Keep the PC, ESP32-S3, data USB cable, and Wi-Fi details nearby. Open one terminal for Flask and one for Axum. Leave room for a third terminal when the serial monitor appears.

There is no prize for rushing. At the end of each stage, look for the small piece of evidence named on the page. If it is missing, stay with that stage.

Sec 10.2 Stage one: wake both PC services

Open `.env` and check the PC's LAN address, Wi-Fi name, Wi-Fi password, and device name. Then start the project:

```
# Windows
.\run-dev.bat

# Linux or macOS
./run-dev.sh
```

Open these two addresses:

```
http://localhost:5000
http://localhost:7000/api/ota/status
```

What to notice: the first tab shows the dashboard; the second says the OTA service is online and prints the correct LAN-facing public URL.

Sec 10.3 Stage two: give the old CSV path a quick health check

Upload the small `sensor-readings.csv` from Chapter 4. Watch progress, open the report, and run a playground preview. Visit Previous Runs and make sure the session is listed.

What to notice: switching to OTA Firmware and back does not lose the CSV session. This proves our newer pages still respect the original Flask work.

Sec 10.4 Stage three: pack the basic application

On Windows PowerShell:

```
1 Compress-Archive \
2   -Path examples\uploaded-firmware-basic\* \
3   -DestinationPath uploaded-firmware-basic.zip
```

Open the ZIP. Its first level should show `Cargo.toml`, `ota-app.toml`, and `src`.

Go to OTA Firmware. Choose the ZIP, select ESP32-S3, type version 1.0.1, and press Upload & Build.

What to notice: the badge moves from uploading to queued and building. The terminal begins with tool versions and the managed-application mode. Real compiler lines follow.

If the build fails, keep the record. Find the first useful error and fix that condition. Success should never be invented by editing SQLite or dropping a random binary into `firmware-builds`.

Sec 10.5 Stage four: inspect what a successful build made

When the badge reaches success, look at the firmware table. The new row should have version 1.0.1, target ESP32-S3, a filename, byte size, SHA-256, time, and Ready status.

Behind the screen, you will find:

- the extracted upload under `firmware-projects`;
- a generated managed workspace inside that project;
- the application binary under `firmware-builds`;
- matching project, build, and firmware rows in `data/ota.db`.

Press Download once. The downloaded byte count should match the row in the table.

Sec 10.6 Stage five: give the board its first base program

This is the physical boundary. Review `examples/esp32-rust-ota-client/partitions.csv` and compare it with the board's flash size. Connect the board with a data cable.

Set the runtime values and flash:

```
1 cd examples\esp32-rust-ota-client
2 $env:WIFI_SSID="your-ssid"
3 $env:WIFI_PASS="your-password"
4 $env:OTA_SERVER_URL="http://192.168.1.5:7000"
5 $env:DEVICE_NAME="Tutorial sensor"
6 cargo +esp run --release --target xtensa-esp32s3-espidf
```

Replace the sample IP before pressing Enter.

Pause before you continue

The command writes to the connected board. Confirm the chip, serial port, flash size, partition table, and stable power first. Do not reuse an example partition map on an existing product until its data partitions have been reconciled.

What to notice: the serial terminal shows startup and Wi-Fi connection. Soon afterward, the board appears on the Devices page with its current version and IP address.

Sec 10.7 Stage six: leave the update note

Open Devices and press Deploy OTA for the new board. Choose the firmware row for 1.0.1.

The card should first say Pending. The runtime checks every thirty seconds, so give it a moment. Then watch Downloading, Verifying, Installing, and Rebooting.

The serial monitor may disappear briefly during restart. Open it again if needed. The new image runs application setup, marks its slot valid, and starts the uploaded main loop.

What to notice: the device card reaches Success and its current version becomes 1.0.1. The serial output contains the basic application's cycle message.

Sec 10.8 Stage seven: change something you can recognize

Copy the basic application. Change its log line to a message you will recognize immediately, such as the room or sensor name. Build it as 1.0.2, then deploy it.

After reboot, look for the new message in the serial monitor. This is the satisfying moment: source from your ZIP is now running inside the complete firmware image built by the PC.

If you want to try dependencies next, use `examples/uploaded-firmware-with-libs`. It already demonstrates `heapless`, `Serde`, and `JSON`.

Sec 10.9 Write down the evidence while it is fresh

A short lab note can save an afternoon later. Record the build ID, firmware ID, version, byte size, hash, device ID, old and new versions, board model, tool versions, and final serial message. Add the real error and solution when a stage needed repair.

The whole path

You have completed the lab when the CSV report still works, a real toolchain build creates firmware, a physically provisioned device registers, deployment becomes an update manifest, the device streams and verifies the binary, and the rebooted image reports version 1.0.1 or later.

11

WHEN SOMETHING GOES WRONG

Sec 11.1 Sit with the last thing that worked

An error becomes easier when you give it an address. Ask one question: what was the last step that definitely worked?

If the dashboard never opened, stay with Flask. If CSV works and the OTA light is dark, stay with Axum. If Axum accepted the ZIP and Cargo failed, stay with the build log. If the PC can download the binary and the board cannot, stay with the network between them.

This habit is slower for ten seconds and faster for the next hour.

Sec 11.2 The dashboard page will not open

Look at the Flask terminal. If it names a missing package, install `python/requirements.txt` into `python/.venv`. Then try:

```
http://127.0.0.1:5000/
```

If that address opens, Flask is serving the page. If the page has no style or buttons do nothing, open `/style.css` and `/app.js` on the same port to see whether those files are served.

Do not open `index.html` by double-clicking it. The resulting `file:///` page lives in a different browser world from the Flask-served dashboard.

Sec 11.3 CSV works and OTA says offline

Open:

```
http://localhost:7000/api/ota/status
```

If it fails, read the Axum terminal. Cargo may still be compiling. Configuration may contain a bad port or URL. Another program may already own port 7000.

If the status opens in its own tab but the dashboard request is blocked, open the browser's Network panel and look for CORS. Compare the dashboard's exact origin with `OTA_ALLOWED_ORIGINS`.

Sec 11.4 The ZIP is turned away

Read the error body before repacking anything. It usually points to one of these:

- the chosen file is not a ZIP;
- `Cargo.toml` is missing;
- `ota-app.toml` uses an old API version;
- `src/main.rs` is missing or the entrypoint points outside the project;
- a local dependency reaches outside the archive;
- the archive contains a symlink or unsafe path;
- the encoded or extracted size is too large.

Open the ZIP and look at its first level. The clean managed layout is `Cargo.toml`, `ota-app.toml`, and `src`.

Sec 11.5 The build stops before your code appears

The first log lines test `cargo`, `rustc`, and `esptool`. If one is missing, run its `--version` command in the same terminal environment used by Axum.

After installing a tool, close and restart the OTA terminal. A running process keeps the environment it had when it started.

On Windows, `link.exe not found` points to Visual Studio Build Tools with Desktop development with C++. The Espressif toolchain alone does not supply that host linker.

Sec 11.6 Rust gives you a page of compiler errors

Scroll to the first clear error with a source path and line number. Later errors often grow from that first one.

For an uploaded application, check the basic handshake:

```
pub fn setup(context: &mut AppContext) -> anyhow::Result<State>
pub fn main(context: AppContext, state: State) -> anyhow::Result<()>
```

The state type returned by `setup` must match the second main parameter. Take hardware from `context.peripherals`. Check that new libraries support the ESP32-S3 ESP-IDF target.

Make one correction and build again. Changing five things at once can hide which one mattered.

Sec 11.7 Cargo succeeds and image creation fails

Find the logged ELF path. The next lines should show the `esptool save-image` stage. Run `esptool --version` in the Axum terminal and confirm the expected command is available.

The server will only accept a non-empty application image in `firmware-builds`. A successful Cargo line does not excuse a missing `.bin`.

Sec 11.8 The PC sees firmware and the board does not

Look at `public_base_url` in the status response. It must contain the PC's LAN IP. Then try that LAN URL from a phone or another computer on the same Wi-Fi.

If the second device cannot open it, check the PC firewall, network profile, VPN routes, and access-point client isolation. If the second device can open it, compare the exact URL compiled into the ESP32 runtime.

Sec 11.9 The device card never appears

The dashboard does not discover USB devices. Registration must come from the running ESP32 program.

Open the serial monitor. Look first for Wi-Fi connection. Then look for the registration POST. Confirm that the board received the right SSID, password, and LAN server URL during its build.

If Wi-Fi provisioning has never happened, return to the first USB flash in Chapter 7.

Sec 11.10 The card stays on Pending

Pending means Axum saved the deployment. It has not heard completion from the device.

Check `last_seen`. The runtime normally polls every thirty seconds. Make sure the desired version differs from the current version and the device chip matches the firmware target. The serial terminal should show the next update check or its network error.

Sec 11.11 Size or SHA-256 does not match

The runtime aborts the inactive writer. That is the safe response. A size mismatch suggests a truncated download or wrong metadata. A hash mismatch means the received bytes do not match the artifact Axum recorded.

Download the same firmware on the PC, inspect the file size, and compare its hash with the metadata. Do not remove the device-side check to make the update continue.

Sec 11.12 The binary has outgrown its drawer

The sample OTA slots are each 0x1e0000 bytes. Compare the binary size with the real partition size. Reduce code or dependencies first. A larger slot requires a carefully revised partition table and usually a new serial provisioning step.

Sec 11.13 Delete answers with 409 Conflict

The firmware still belongs to a device's desired state or deployment history. The refusal protects those links. Do not delete the binary by hand while SQLite still points to it.

Sec 11.14 Where are the ESP32 terminal messages?

They are still in the serial monitor. Build History shows PC compiler output. Devices shows heartbeats and OTA status. Current dashboard code does not stream board logs.

Use `espflashmonitor` today. The next chapter shows how a future serial bridge or network logger could bring those messages into a clearly labeled dashboard panel.

Try it before moving on

Take one real failure from your terminal. On paper, write the last successful step, the failing boundary, the smallest test for that boundary, and one change to try. Keep the original error beside the result.

12

WHERE YOU CAN TAKE IT NEXT

Sec 12.1 Choose one seam and keep the rest quiet

The project is already divided along useful seams. A good extension changes the smallest responsible piece and lets its neighbors keep their familiar contracts.

Before writing code, finish this sentence: “The new behavior belongs to _____ because that piece already owns _____.”

If the answer is Wi-Fi credentials, open `wifi-manager`. If it is a new HTTP address, open `routes`. If it is a new build environment, open the `FirmwareBuilder` implementation.

Sec 12.2 Welcome another ESP32 chip

The current board list contains ESP32-S3. A second target needs more than another dropdown option.

Add its accepted name in target validation. Give `TargetSpec` the proper Rust target and `espflash` chip. Add the board choice in the frontend. Then make a runtime peripheral split that matches the actual chip, partition example, and rollback configuration.

An ESP32-C3, for example, uses RISC-V and does not offer the same hardware bundle as the S3. The application API should promise only resources the board truly has.

Begin with a known-good empty application. Prove build and serial flash before testing OTA and rollback.

Sec 12.3 Turn Wi-Fi setup into a friendly first-run flow

The current manager receives credentials at build time and reconnects when needed. A future board could behave like a new home device:

1. Look for saved credentials in NVS.
2. Try the saved network for a limited time.
3. Start its own temporary setup access point when no network works.
4. Show a small page where the owner chooses Wi-Fi.
5. Save the credentials and return to station mode.
6. Publish each change through the existing network handle.

Keep the modem, event loop, NVS, and Wi-Fi service inside `wifi-manager`. The sensor application should not have to learn captive-portal code.

Sec 12.4 Bring the ESP32 terminal into the dashboard

There are two useful roads, and they show different parts of the journey.

12.4.1 Road one: a serial bridge on the PC

A small local service opens a selected COM or tty port, reads lines, and streams them to the browser with SSE or WebSocket. It can show bootloader messages, panics, and Wi-Fi failures before the network comes alive.

The dashboard must ask the user which port to open. Only one reader should own that port. The bridge needs bounded memory, clean disconnects, and a visible warning when flashing cannot begin because monitoring still owns the serial connection.

12.4.2 Road two: logs sent over Wi-Fi

The runtime can gather structured log records, post small batches to Axum, and let the browser subscribe to a device stream. This works after the USB cable is gone. It cannot report a failure that prevents Wi-Fi from starting.

A good dashboard labels each line as Serial or Network. The server also needs retention limits so a noisy loop cannot fill SQLite or memory.

What exists today

Neither device-log road is implemented in the current API. The existing dark terminal is only for PC build logs. Keeping that sentence visible prevents a mock terminal from being mistaken for hardware evidence.

Sec 12.5 Move untrusted builds into a safer room

Today `LocalFirmwareBuilder` runs Cargo on the host PC. That is suitable for trusted local projects. Cargo can execute build scripts and procedural macros, so an unknown ZIP deserves a stronger boundary.

The `FirmwareBuilder` trait leaves a doorway for `DockerFirmwareBuilder`. A careful container would use a pinned toolchain image, a read-only source mount, a small writable output folder, and limits for time, CPU, memory, processes, and logs. It would carry no host secrets.

The coordinator can keep receiving the same log lines and final artifact. Routes and frontend polling do not need to learn how the build room changed.

Sec 12.6 Give firmware a real signature

Keep SHA-256 for transfer integrity. Add publisher proof as a separate layer built from supported signing tools and device trust anchors.

The build system would sign an approved image or digest with a protected private key. Firmware metadata would carry the signature, algorithm, and key ID. The device would verify them before activating the slot. Key rotation and recovery need their own tested plan.

Use ESP-IDF Secure Boot and signing guidance when moving toward production. A homemade shared-secret formula is too fragile for the job.

Sec 12.7 Let the API grow without surprising old projects

Managed uploads already declare `application_api_version=2`. When the application contract changes, add or migrate versions deliberately. Give an old ZIP a clear rejection message and a short conversion example.

Prefer adding fields that old code can ignore. Keep runtime and application API versions pinned together in the generated workspace. If public HTTP payloads need a breaking change, give the new contract a visible version boundary.

Sec 12.8 Test from the cheap layer toward the board

Start with unit tests for paths, versions, manifest reading, and status changes. Add integration tests around a temporary SQLite database and a fake builder. Check frontend navigation and failure states in the browser. Then run a known-good project through the real ESP toolchain.

Physical tests come last because they take the most time and reveal a different class of problem. Test a successful update, unplugged network, interrupted download, wrong hash, power loss, setup failure, rollback, and repeated slot swapping.

Write down which layers passed. “The Rust tests pass” and “rollback worked on this board” are different pieces of evidence.

Try it before moving on

Pick one extension from this chapter. Fill in the ownership sentence at the beginning. Below it, draw the files you would touch, the new failure a learner would see, and the smallest test that proves the change without hardware. Add the physical test on the last line.



HTTP API REFERENCE

This appendix is a desk reference. You do not need to read it from top to bottom before using the project. Return here when you are writing a request, checking a field name, or wondering which service owns an address. The story behind these calls lives in Chapters 3 through 9.

Sec A.1 Conventions

The dashboard uses JSON for ordinary requests and responses, multipart form data for project upload, server-sent events for CSV progress, HTML for profiling reports, and raw octet streams for firmware downloads.

OTA failures share this envelope:

```
{
  "success": false,
  "error": {
    "code": "TARGET_MISMATCH",
    "message": "Device chip does not match firmware target",
    "details": "optional diagnostic detail",
    "hint": "optional corrective action"
  }
}
```

Fields with no value may be omitted. IDs are UUID strings except `device_id`, which is chosen by configuration or derived from the station MAC.

Sec A.2 OTA endpoint index

Method	Endpoint	Purpose
GET	/api/ota/status	Service health and public URL
GET	/api/ota/projects	List uploaded projects
POST	/api/ota/projects	Upload a project ZIP
POST	/api/ota/projects/<project-id>/build	Queue a build
GET	/api/ota/builds	List build history
GET	/api/ota/builds/<build-id>	Build status and full log
GET	/api/ota/firmware	List ready firmware
GET	/api/ota/firmware/latest?target=esp32s3	Newest ready target image
GET	/api/ota/firmware/<firmware-id>	Firmware details
GET	/api/ota/firmware/<firmware-id>/download	Stream application image
DELETE	/api/ota/firmware/<firmware-id>	Delete unreferenced image
GET	/api/ota/devices	List registered devices
POST	/api/ota/devices/register	Create or update a device
POST	/api/ota/devices/<device-id>/heartbeat	Refresh device state
POST	/api/ota/devices/<device-id>/deploy	Select desired firmware
GET	/api/ota/devices/<device-id>/update	Poll desired firmware
POST	/api/ota/devices/<device-id>/ota-status	Report OTA progress/result

Sec A.3 Status

GET /api/ota/status returns 200.

```
{
  "service": "CSV_PROFILING OTA Server",
  "language": "rust",
  "status": "online",
  "version": "0.1.0",
  "public_base_url": "http://192.168.1.5:7000",
  "supported_targets": ["esp32s3"]
}
```

Sec A.4 Projects and builds

POST /api/ota/projects expects multipart fields `project`, `target`, and `version`. A successful upload returns 201.

```
curl -F "project=@uploaded-firmware-basic.zip" \
-F "target=esp32s3" -F "version=1.0.1" \
http://localhost:7000/api/ota/projects
```

```
{
  "project_id": "55e4...",
  "name": "uploaded-firmware-basic",
}
```

```

    "target": "esp32s3",
    "version": "1.0.1",
    "status": "uploaded",
    "project_type": "managed_application",
    "application_api_version": 2,
    "entrypoint": "src/main.rs"
  }

```

Standalone uploads omit the API version and entrypoint and report `project_type` as `standalone`.

GET `/api/ota/projects` returns an array of project records. Internal filesystem paths are skipped during serialization.

POST `/api/ota/projects/<project-id>/build` returns 202:

```

{"build_id": "a8c4...", "status": "building"}

```

The durable row begins queued and is moved to building by the coordinator task. Only one queued or active build is allowed for a project.

GET `/api/ota/builds` returns all builds, newest first. **GET** `/api/ota/builds/<build-id>` returns one response shaped like:

```

{
  "build_id": "a8c4...",
  "project_id": "55e4...",
  "project_name": "uploaded-firmware-basic",
  "target": "esp32s3",
  "version": "1.0.1",
  "status": "success",
  "logs": ["Build queued", "Build started", "Tooling: cargo ..."],
  "exit_code": 0,
  "error": null,
  "started_at": "2026-09-07T10:00:00Z",
  "finished_at": "2026-09-07T10:02:00Z",
  "created_at": "2026-09-07T09:59:59Z"
}

```

Failed responses may include `hint`.

Sec A.5 Firmware

List, detail, and latest routes return the same firmware representation:

```

{
  "firmware_id": "c53b...",
  "project_id": "55e4...",
  "build_id": "a8c4...",
  "project": "uploaded-firmware-basic",
  "target": "esp32s3",
  "version": "1.0.1",
  "filename": "firmware-uploaded-firmware-basic-v1.0.1-a8c4.bin",
  "size": 945332,
  "sha256": "f2534c...",
  "status": "ready",
  "created_at": "2026-09-07T10:02:00Z",
}

```

```
{
  "download_url": "http://192.168.1.5:7000/api/ota/firmware/c53b.../download"
}
```

GET /download returns `application/octet-stream`, content length, attachment disposition, and `X-Firmware-SHA256`. The route reconstructs the path from the configured build directory and stored filename, then canonicalizes it before opening.

DELETE /api/ota/firmware/<firmware-id> returns 204. It returns 409 when a device or deployment references the image.

Sec A.6 Devices and deployments

POST /api/ota/devices/register returns 201 and the current device record.

```
{
  "device_id": "ESP32-S3-001",
  "name": "Sensor Node 1",
  "chip": "esp32s3",
  "current_version": "1.0.0",
  "ip_address": "192.168.1.24"
}
```

POST /api/ota/devices/ESP32-S3-001/heartbeat returns 200 and the updated device.

```
{
  "current_version": "1.0.0",
  "status": "online",
  "ip_address": "192.168.1.24"
}
```

Allowed heartbeat statuses are online, offline, updating, rebooting, updated, failed, and unknown.

GET /api/ota/devices returns records with these public fields:

```
{
  "device_id": "ESP32-S3-001",
  "name": "Sensor Node 1",
  "chip": "esp32s3",
  "ip_address": "192.168.1.24",
  "current_version": "1.0.0",
  "desired_version": "1.0.1",
  "desired_firmware_id": "c53b...",
  "status": "online",
  "ota_status": "pending",
  "ota_progress": 0,
  "ota_error": null,
  "last_seen": "2026-09-07T10:03:00Z"
}
```

POST /api/ota/devices/<device-id>/deploy expects a firmware ID and returns 202.

```
{"firmware_id": "c53b..."}
```

```
{
  "deployment_id": "d911...",
}
```

```

    "device_id": "ESP32-S3-001",
    "firmware_id": "c53b...",
    "desired_version": "1.0.1",
    "status": "pending"
  }

```

GET `/api/ota/devices/<device-id>/update` returns either `update_available:false` or the manifest shown in Chapter 9.

POST `/api/ota/devices/<device-id>/ota-status` accepts:

```

{
  "status": "downloading",
  "progress": 35,
  "current_version": null,
  "error": null
}

```

Allowed OTA states are idle, pending, downloading, verifying, installing, rebooting, success, and failed. Progress must be between 0 and 100. Failed requires an error. Success defaults to 100 and clears desired firmware after updating the current version.

Sec A.7 CSV endpoint index

These routes belong to Flask on port 5000.

Method	Endpoint	Purpose
GET	/	Serve the dashboard HTML
POST	/upload	Accept multipart CSV field <code>file</code> and return <code>job_id</code>
GET	/progress/<job-id>	Stream profiling progress with SSE
GET	/report/<job-id>	Serve generated report HTML
GET	/sessions	List retained report sessions
GET	/session/<job-id>	Check and restore one session
POST	/clean/<job-id>	Apply cleaning controls and refresh meta-data
POST	/playground/<job-id>	Run preview, analysis, chart, or explicit mutation
GET	/download-cleaned/<job-id>	Download cleaned CSV
DELETE	/reset/<job-id>	Delete one CSV session
DELETE	/reset-all	Delete all CSV sessions

Playground request fields include `operation`, `mutate`, `limit`, `column`, `column_2`, `value`, `operator`, `indexes`, and `chart_type`. The endpoint clamps limit to 1 through 500.



CONFIGURATION, COMMANDS, AND GLOSSARY

Keep this appendix near your terminal. It gathers names and commands that are easy to forget after you understand the larger idea. Copy a command only after reading the short note around it, especially when hardware is connected.

Sec B.1 Environment variables

Variable	Default/example	Meaning
OTA_BIND_ADDRESS	0.0.0.0	Listener interface; LAN access needs a non-loopback binding.
OTA_PORT	7000	Axum HTTP port.
OTA_PUBLIC_BASE_URL	detected or explicit	Base placed in device-facing firmware URLs. Use the PC LAN address.
OTA_PROJECT_DIR	firmware-projects	Safe root for extracted uploads.
OTA_BUILD_DIR	firmware-builds	Safe root for generated application images.
OTA_DATA_DIR	data	Directory containing <code>ota.db</code> .
OTA_RUNTIME_DIR	examples/esp32-rust-ota-client	Stable runtime template used for managed builds.
OTA_MAX_UPLOAD_MB	50	Encoded project field limit in MiB.
OTA_MAX_EXTRACTED_MB	upload limit times 10	Maximum total declared extracted content.
OTA_ALLOWED_ORIGINS	localhost and 127.0.0.1 on port 5000	Comma-separated browser origin allow-list.
OTA_RUST_TOOLCHAIN	esp	Rustup toolchain applied to builder commands; empty uses the process default.
WIFI_SSID	none	Embedded Wi-Fi network name used at managed build time.
WIFI_PASS	none	Embedded Wi-Fi password. Keep it in local <code>.env</code> .
DEVICE_NAME	managed-device label	Friendly device name embedded into managed firmware.

Variable	Default/example	Meaning
DEVICE_ID	MAC-derived when absent	Optional stable identity override.
RUST_LOG	info defaults	Axum and tower-http tracing filter.

Sec B.2 Command sheet

B.2.1 Start services

```
# Windows
.\run-dev.bat

# Linux/macOS
./run-dev.sh
```

B.2.2 Run OTA server checks

```
cargo test --manifest-path ota-server/Cargo.toml
cargo clippy --manifest-path ota-server/Cargo.toml --all-targets
curl http://localhost:7000/api/ota/status
```

B.2.3 Check embedded tools

```
cargo --version
rustc +esp --version
espflash --version
rustc +esp --print target-list
```

B.2.4 Build the stable runtime without flashing

```
cd examples/esp32-rust-ota-client
cargo +esp build --release --target xtensa-esp32s3-espidf
```

B.2.5 Build and flash the stable runtime

```
cargo +esp run --release --target xtensa-esp32s3-espidf
```

The runner settings determine the exact `espflash` arguments and partition-table use. Inspect `.cargo/config.toml` in the embedded project before physical flashing.

Sec B.3 Glossary

- Application image** A `.bin` containing one ESP-IDF application, suitable for an OTA app partition.
- Axum** The Rust HTTP framework used by the OTA service.
- Build ID** UUID identifying one compilation attempt and its durable logs.
- Cargo target** The architecture and platform a Rust crate is compiled for. This project begins with `xtensa-esp32s3-espidf`.
- CORS** Browser policy controlling which page origins may call the OTA API.
- Deployment** A stored relationship saying one device should install one firmware record.
- ELF** Linked executable format consumed by `espflashsave-image`.
- ESP-IDF** Espressif's official development framework and system services used beneath Rust bindings.
- Firmware ID** UUID identifying one ready application image and metadata record.
- Flask** The Python HTTP framework serving the dashboard and CSV API.
- Heartbeat** Periodic device request refreshing version, status, IP address, and last-seen time.
- Inactive slot** OTA application partition that is not currently executing and can safely receive a new image.
- Managed application** Uploaded function-based Rust package composed with the stable runtime by the server.
- NVS** ESP-IDF non-volatile storage, commonly used for settings and Wi-Fi configuration.
- OTA** Over-the-air firmware update performed through a network connection.
- Pull model** The device polls for desired firmware and initiates its own download.
- Rollback** Bootloader-assisted return to a previously valid application slot when a new image fails startup validation.
- SHA-256** Hash used here to confirm downloaded bytes match server metadata.
- SSE** Server-sent events, the one-way HTTP stream used for CSV profiling progress.
- Stable runtime** Initially flashed platform code that owns Wi-Fi, OTA, rollback, and application lifecycle.
- Standalone firmware** Uploaded Cargo project built without managed-runtime composition; it owns its complete device behavior.
- WAL** SQLite write-ahead logging mode, used to improve local concurrent access behavior.
- ZIP slip** Archive path traversal that attempts to extract outside the intended root.

Sec B.4 Current implementation limits

The first version supports ESP32-S3, local SQLite storage, polling for build logs, plain HTTP on a trusted LAN, local trusted builds, and SHA-256 integrity checks. Device serial/network terminal streaming, isolated Docker builds, multi-user authentication, publisher signing, and additional chip targets are extension work described in Chapter 12.

These limits are design boundaries, not hidden promises. Keeping them visible helps a beginner distinguish what can be used today from what still needs engineering and hardware validation.

CLOSING NOTE

You now have a map from browser click to Pandas report, and from uploaded Rust source to an application image written into an inactive ESP32 OTA slot. That map matters more than memorizing a command. When something fails, locate the boundary first: browser to Flask, browser to Axum, builder to toolchain, PC to LAN, or bootloader to application. The logs on that boundary usually tell you what to fix next.

Keep the initial runtime recoverable over USB, test rollback on real hardware, and treat uploaded build inputs as trusted code until the builder runs inside a sandbox. With those habits, this project can grow without losing the clean separation that makes it understandable.